



Running PostgreSQL on Kubernetes in Production

Day 2 Operations with CloudNativePG

Michael Raeck

ContainerDays Hamburg · September 2026





\$whoami

Michael Raeck

"Hands on" CTO, Paladyn · venture builder (Woonzorgweb, ~150k users/month)

Independent Platform Engineer, Vantralis

Kubernetes, GitOps & CloudNativePG in ISO 27001-certified environments

100+ CNPG databases · bucket + user + backup schedule generated per DB CR



CloudNativePG, August 2026

What is current, and what is already end of life.

1.30.0	current stable, 29 June 2026
1.29.x	supported until 29 Sep 2026 , four weeks from now
1.28 / 1.27 / 1.26	end of life
PostgreSQL 14–18	18.4 at 1.30.0's release · 18.6 current (18.5 never shipped)
Kubernetes 1.34–1.36	

⚠ **Three CVEs.** CVE-2026-44477 , CVSS 9.4, **metrics exporter** (fixed 1.29.1 / 1.28.3). CVE-2026-55769 privilege escalation via the public schema, and CVE-2026-55765 cleartext passwords on CREATE/ALTER ROLE , both fixed in **1.30.0**, backported to 1.29.2 / 1.28.4.



The Operator Is the Easy Part

One CR gets you a cluster. Everything that hurts afterwards lives underneath it.

Storage — the IOPS you did not choose, and the WAL volume you did not separate

Object store — it decides how long recovery takes, not Postgres

Topology — where instances land, and which of them can become primary

Every demo after this is one of those three, breaking.



Let's deploy

One Cluster CR, three instances

Apply · the operator elects a primary · connect
through PgBouncer



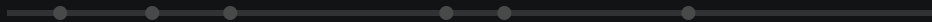


Demo 1: Deploy

```
> # 30-ish lines of YAML. One Cluster CR.  
> bat --play -
```

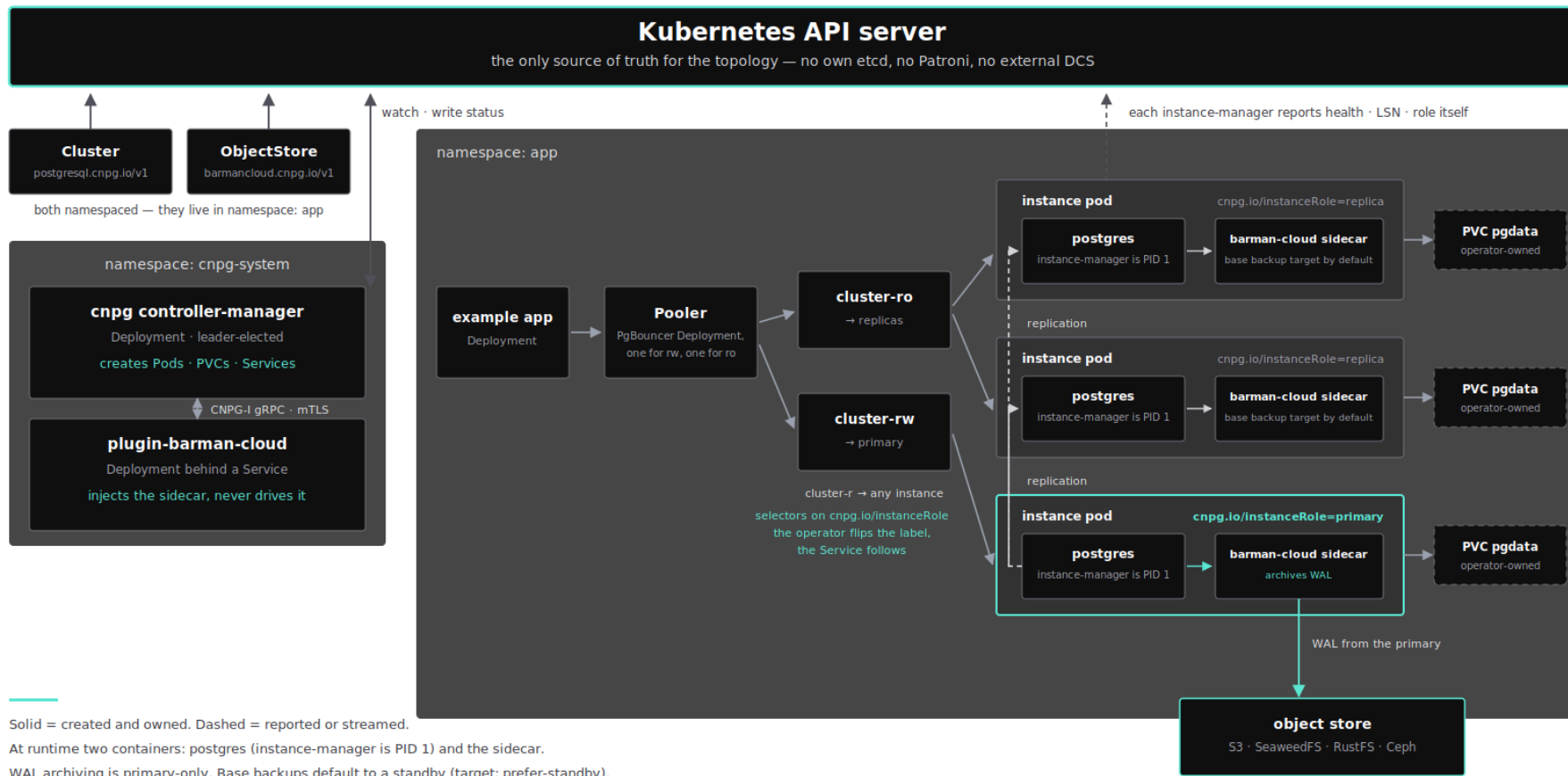


00:00





The Whole Architecture





Day 0 Decides Your Day 2

You just watched me deploy `storage: 2Gi` with no storageClass.

Volume	Published performance
OVH Classic / Regional Classic	guaranteed 500 IOPS , 64 MB/s
OVH High Speed Gen2	30 IOPS/GB → max 20,000 · 0.5 MB/s/GB → max 320 MB/s
Scaleway Block Storage	up to 15,000 IOPS , "subject to fluctuation during peak load"
Hetzner Cloud Volumes	not published . "fast (SSD based)"
AWS gp3	3,000 IOPS + 125 MiB/s baseline, no bursting
AWS gp2	3 IOPS/GiB → 100 GiB = 300 IOPS, bursts, then 5 h to refill credits

IOPS is often a function of volume size: OVH Gen2 30/GB, gp2 3/GiB. A small volume is a slow database, and it benchmarks fine until the burst credits run out.

✓ Separate `walStorage` · `allowVolumeExpansion` · `WaitForFirstConsumer` ✗ Burstable volumes for a database you must give an RTO for



Let's back it up

An ObjectStore and a sidecar

Wire the bucket · a sidecar appears · one backup
now, one every night



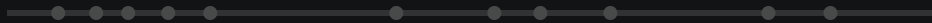


Demo 2: ObjectStore + Backup

```
> # Plugin runs cluster-wide, installed by 00-setup. Not baked into the op
```

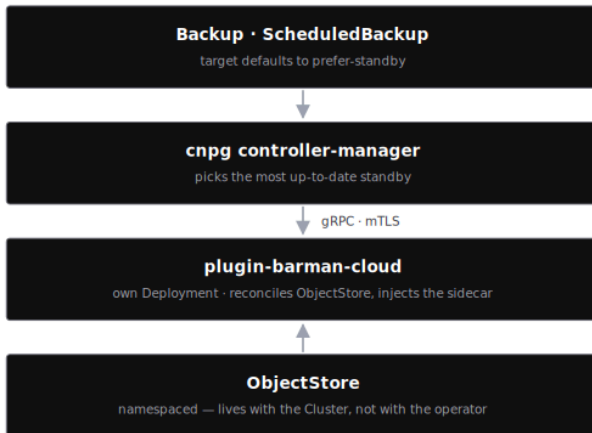
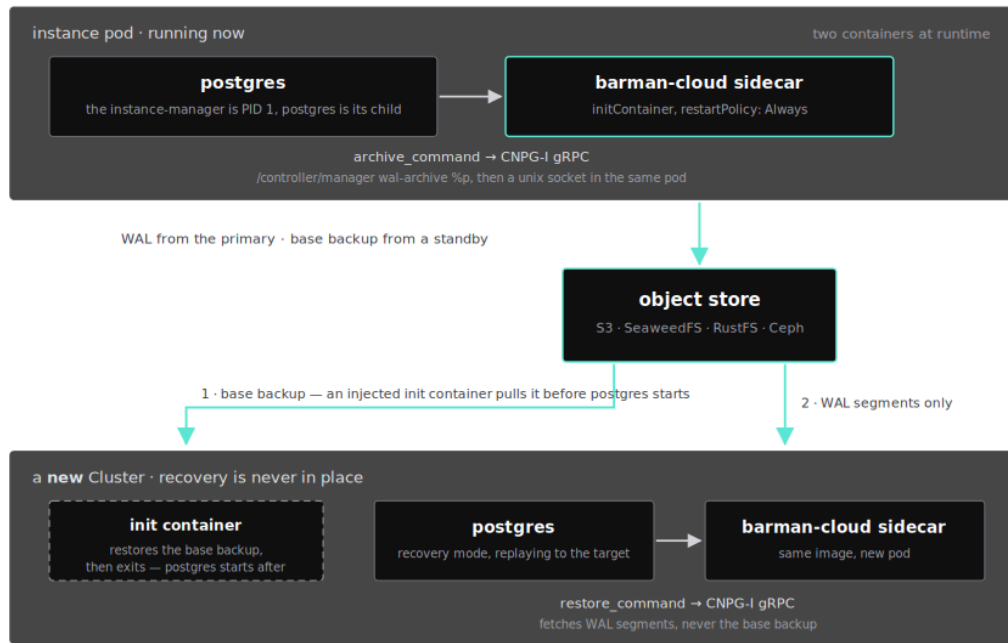


00:00





How Backups Actually Work



The base backup runs in the sidecar, next to the data — on a standby by default, on the primary only as a fallback.

WAL archiving is the exception: primary only.

Credentials stay in the Cluster's namespace.

Why a sidecar and not the operator: **IRSA per cluster · blast radius · version skew**



Time to Recovery

count in Network to your TTR.

Database	25 MB/s cross-region	100 MB/s	250 MB/s	500 MB/s same DC
10 GB	7 min	2 min	41 s	20 s
100 GB	68 min	17 min	7 min	3 min
500 GB	5.7 h	85 min	34 min	17 min
1 TB	11.7 h	2.9 h	70 min	35 min

+ all Wal Replays since last Base Backup

gZip should be used, but only swaps Transport for CPU, but this can still be faster

can be tweaked: `ObjectStore` , `data.jobs` and `wal.maxParallel` .

Measure this: The whole Recovery procedures require this



A Major Upgrade Does Three Things

Only the first is fast, and the first is the only one anybody quotes

Step	Scales with	On a toy DB	On 1 TB
<code>pg_upgrade --link</code>	table count (hard links)	seconds	seconds–minutes
statistics transfer <i>(new in 18)</i>	table + column count	seconds	can triple the whole job
replica rebuilds via <code>pg_basebackup</code> ×N	data size ÷ min(net, disk)	seconds	see the previous slide

A replica is re-cloned **from the primary, not the bucket**, so it costs read IOPS on a production database, write IOPS on a fresh volume, and **HA is degraded for the entire copy**.

PG 18 preserves optimizer statistics. A real win, and it is not free at upgrade time. `--no-statistics` opts out if you would rather pay it afterwards.



Let's break it

At ContainerDays. On purpose.

One UPDATE without a WHERE · recover · then
repair production



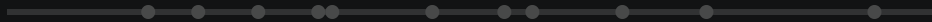


Demo 3: Point-in-Time Recovery

```
> # What is actually in the harbour right now.  
> PGPW $(
```



00:00





PITR: the part people get wrong

There is no in-place undo. Recovery is **always** a new cluster, then a reconcile.

```
bootstrap:
  recovery:
    source: hafen-source
    recoveryTarget:
      targetTime: "2026-09-03 10:42:17+00"
```

Target	Use when
targetTime	you know roughly when it happened
targetLSN	you know exactly where in the WAL
targetXID	you have the offending transaction id
targetName	you called <code>pg_create_restore_point()</code> before the risky change

Restore time = fetch base backup + replay WAL since that backup. The second term grows with time-since-backup, so backup frequency is an **RTO** decision.



Let's upgrade it

Postgres 17 to 18, in
place, one field

One field · `pg_upgrade --link` · replicas re-
cloned · then the statistics



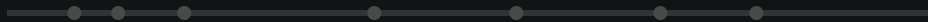


Demo 4: In-Place Major Upgrade, 17 → 18

```
> # Where are we today?  
> kubectl -n hafen get cluster h
```



00:00





Let's fail over

A replica cluster, and
one field

Restore from the same bucket, forever · read only ·
then promote





Demo 5: Replica Cluster for DR

```
> # Same bucket. New namespace. replica.enabled: true.
```



00:00





Where Do Instances Actually Land?

Spreading is a *hint*, not a guarantee

```
spec:
  affinity:
    enablePodAntiAffinity: true           # default
    podAntiAffinityType: preferred        # default ← a hint
    topologyKey: kubernetes.io/hostname  # default
```

5 instances, 3 worker nodes. What actually happened:

```
hafen-2   hafen-worker3   hafen-4   hafen-worker
hafen-3   hafen-worker    hafen-5   hafen-worker3 ← doubled up
```

- ❌ `podAntiAffinityType: required` → pods **Pending forever** when instances > nodes
- ✅ `spec.topologySpreadConstraints`, top level, expresses `maxSkew`, which anti-affinity cannot



"Can I keep the primary off my edge node?"

No. And the reason is worth understanding.

Primary is a role, not an identity

It moves on failover. Anything on that node is promotable **by definition**.

One pod template for every instance, so `spec.affinity` is **cluster-wide**. No per-instance placement, no pinning. **So an edge/DR site is a second Cluster, not instance #4:**

```
spec:
  replica: { enabled: true, source: hafen-source } # read-only, WAL replay
  affinity:
    nodeSelector: { node-role.kubernetes.io/edge: "" }
    tolerations: [{ key: edge, operator: Exists, effect: NoSchedule }]
```

`syncReplicaElectionConstraint` **does not solve this**: it governs *sync replica election* only; an excluded replica can still be promoted.



Six Gotchas, Found the Hard Way

Sidecar is an *init* container

Pods show 2/2 , but it lives under `.spec.initContainers`

Upgrade Job is `<primary>-major-upgrade`

Not `<cluster>-...` , and **deleted seconds after success**

DR needs a *post-upgrade* base backup

A PG 17 base backup cannot bootstrap PG 18

`--link` **has no rollback**

The old data directory is gone. Your escape is the *pre-upgrade* backup

`retentionPolicy` **lives on**
`ObjectStore`

On `ScheduledBackup` it is a strict decoding error

The operator owns `postgresql.conf`

`spec.postgresql.parameters` is rendered into it. 1.30 validates the keys, because they used to inject arbitrary directives



Takeaways

Your object store is the variable

Not Postgres. Measure against the bucket you will actually restore from.

Backup frequency is an RTO decision

Restore = fetch backup + replay WAL. Only the second term is yours.

Recovery is always a new cluster

There is no in-place undo. Practise it. Use named restore points.

Storage is chosen on day 1, paid for on day 2

IOPS follow volume size. Separate the WAL volume. And demo values are demo values.

Topology is a Cluster boundary

Not an affinity rule. Primary is a role, and it moves.

The sidecar model is the future

In-tree Barman dies in 1.31. Migrate before it is urgent.

Day 2 starts on day 1

One CR gets you a cluster. Everything else is the runbook.



Resources

- cloudnative-pg.io/docs · github.com/cloudnative-pg/cloudnative-pg
- Barman Cloud plugin (CNPG-I): github.com/cloudnative-pg/plugin-barman-cloud
- `ObjectStore` [reference](#), every field on the CR: cloudnative-pg.io/plugin-barman-cloud/docs/object_stores
- [Grafana dashboard ID 20417](#)
- `kubectkl krew install cnpg`

Everything in this talk, reproducible from zero:

github.com/mraeck/containerdays-2026-cnpg

One script builds the kind cluster, the operator, the plugin, the object store and Grafana. The demo drivers re-record every clip you just watched.



Resources

- cloudnative-pg.io/docs · github.com/cloudnative-pg/cloudnative-pg
- Barman Cloud plugin (CNPG-I): github.com/cloudnative-pg/plugin-barman-cloud
- `ObjectStore` [reference](#), every field on the CR: cloudnative-pg.io/plugin-barman-cloud/docs/object_stores
- [Grafana dashboard ID 20417](#)
- `kubectll krew install cnpg`

Everything in this talk, reproducible from zero:

github.com/mixxor/containerdays-2026-cnpg

One script builds the kind cluster, the operator, the plugin, the object store and Grafana. The demo drivers re-record every clip you just watched.

paladın

BACKUP





Monitoring

Instance :9187/metrics · Operator :8080/metrics · Grafana dashboard [20417](#)

Category	Metric	Alert threshold
Health	cnpg_collector_up	= 0 for 1m
Health	cnpg_cluster_ready_instances	< expected for 5m
Replication	cnpg_pg_replication_lag	> 30s warn / > 300s crit
WAL	cnpg_collector_pg_wal_archive_status	failed count > 0
Safety	cnpg_pg_database_xid_age	> 300,000,000
Backup	barman_cloud_cloudnative_pg_io_last_available_backup_timestamp	older than 2× your schedule

This is the dashboard from the run you just watched.



Automated Failover

No Patroni. No Stolon. Instance manager + Kubernetes.

Phase 1: detection & shutdown	Phase 2: promotion
Primary readiness probe fails	Leader election among replicas
Controller marks TargetPrimary pending	Chosen replica promotes
Fast shutdown signal → primary	Former primary rejoins via <code>pg_rewind</code>
WAL receivers on replicas stop	Cluster resumes

```
spec:
  instances: 3
  failoverDelay: 10          # seconds, prevents flapping
  switchoverDelay: 60
  postgresql:
    synchronous:
      method: quorum
      number: 1
      failoverQuorum: true   # stable since 1.28
```



Quorum Failover Maths

$$R + W > N$$

R	promotable replicas reporting state to the operator
W	replicas acknowledging synchronous commits
N	total potentially synchronous replicas

Cluster	Failure	Failover?	Why
3-node, sync=1	primary only	yes	$R=2, W=1, N=2 \rightarrow 3 > 2$
3-node, sync=1	primary + 1 replica	no	$R=1, W=1, N=2 \rightarrow 2 = 2$
5-node, sync=2	primary + 1 replica	yes	$R=3, W=2, N=4 \rightarrow 5 > 4$

`local` and `off` **bypass the guarantee.** `on`, `remote_write` and `remote_apply` all wait for the synchronous standbys. A session doing `SET synchronous_commit TO local` does not.



Storage

CNPG manages PVCs directly, with no StatefulSet

```
spec:
  storage:    { size: 100Gi, storageClass: premium-ssd }
  walStorage: { size: 20Gi,  storageClass: premium-ssd-wal }
```

StorageClass requirements

- ✓ `WaitForFirstConsumer` for zone-aware binding
- ✓ `allowVolumeExpansion`, your only way out of a full disk
- ✓ `reclaimPolicy: Retain` · SSD / NVMe only

Anti-patterns

- ✗ NFS or HDD without IOPS guarantees
- ✗ `ReadWriteMany` · skipping `WaitForFirstConsumer`

Separate the WAL volume, and size it $\geq 3 \times$ `max_wal_size`.



Services & Connection Routing

Three Services per cluster. The operator keeps their endpoints correct

Service	Targets	Use
hafen-rw	the current primary only	all writes
hafen-ro	replicas only	read-only queries
hafen-r	every instance, primary included	"any node will do"

`-ro` has no endpoints when every replica is down. `-r` still resolves.

Pooler mode	Best for	Caveat
transaction	web apps	no session state, no prepared statements
session	legacy apps	far lower connection reuse



GitOps with ArgoCD

```
syncPolicy:
  automated:
    prune: false           # NEVER auto-delete a database
    selfHeal: true
  syncOptions:
    - ServerSideApply=true
```

- `prune: false` : a pruned `Cluster` is a deleted database
- **ServerSideApply**: the operator writes to status/spec; client-side apply fights it
- **SealedSecrets / ESO**: never plain Secrets in Git
- **One Application per cluster**: independent lifecycle and blast radius



Alerting on Those Metrics

Seven PrometheusRules ship with the operator; these are the ones worth tuning

```
groups:
- name: cloudnativepg.rules
  rules:
    - alert: CNPGClusterDown
      expr: cnpg_cluster_ready_instances == 0
      labels: { severity: critical }

    - alert: CNPGClusterHADegraded
      expr: cnpg_cluster_ready_instances < cnpg_cluster_instances
      for: 5m
      labels: { severity: warning }

    - alert: CNPGWALArchiveFailing
      expr: cnpg_collector_pg_wal_archive_status{status="failed"} > 0
      for: 5m
      labels: { severity: critical }
```

WAL archive failure is the one to page on. It silently invalidates every future PITR.



Top Production Issues

Issue	Symptom	Fix
WAL disk exhaustion	primary crash-loops	expand PVC; check for stale replication slots
Replica out of sync	WAL gap, crash-loop	delete pod + PVC → operator re-clones via <code>pg_basebackup</code>
NetworkPolicy blocks operator	"cannot extract Pod status"	allow <code>cnpg-system</code> → port 8000/TCP. Since 1.30 that channel is ECDSA-authenticated (not backported), so on ≤ 1.29 keep restricting it
XID wraparound	<code>xid_age > 300M</code>	<code>VACUUM (FREEZE, VERBOSE)</code> on the affected table
HugePages mismatch	bus error, exit 135	<code>huge_pages: try</code> , or <code>hugepages-2Mi ≥ shared_buffers</code>
OOMKill	exit 137, no PG error	memory requests = limits; recheck <code>shared_buffers + work_mem × conns</code>

```
kubectl cnpg status hafen -n hafen
kubectl cnpg report cluster hafen -n hafen --logs # everything, zipped
```